

ASSIGNMENT 03

Smart Home

Analisi dei requisiti

Il progetto richiede la modellazione di un sistema di allarme domestico reattivo realizzato interamente tramite attori e scambio dei messaggi. Il sistema deve monitorare l'ambiente, gestire transizioni temporali e reagire agli input dell'utente (tramite l'inserimento di un PIN).

La centrale di allarme reagisce in modo diverso a seconda del suo stato attuale. Quando il sistema è disattivato, o in attesa che l'utenza esca dalla casa, gli eventi rilevati dai sensori vengono ignorati. Il sistema può essere attivato inserendo il PIN corretto tramite la tastiera. Per evitare falsi allarmi, il sistema deve gestire i ritardi temporali sia durante la fase di uscita che durante quella di ingresso. Gli stati che il sistema supporta sono i seguenti:

- **Disarmed:**
 - ignora i messaggi dai sensori
 - Messaggio positivo da user Interaction. Passa allo stato "Exit delay"
- **Exit Delay:**
 - Manda un messaggio a se stesso usando un timers per un messaggio con un delay di un certo numero di secondi
 - Aspetta il messaggio "FinisciTimer()", passando allo stato **Armed**
- **Armed:**
 - manda un messaggio "AlarmStarting()" all'Attore Alarm se riceve un messaggio di "MotionDetected(sensorID: UUID)" da un qualsiasi sensore ed entra in **Entry Delay**
- **Entry Delay:**
 - se riceve un messaggio da UserInterface manda un messaggio "TurnOff()" all'Attore Alarm, e setta lo stato su **Disarmed**
 - se riceve il messaggio "Alarm" manda il messaggio "TurnOff()" all'Attore Alarm ed entra in stato **Emergency state**
- **Emergency state:**
 - suona la sirena
 - se riceve un messaggio positivo da UserInteface, lo stato cambia a **Disarmed**

Per questo tipo di problema abbiamo adottato l'approccio ad Attori, un meccanismo sicuro e moderno per far parlare tra di loro i processori, che non introduce le vulnerabilità generate dalla programmazione multithreading. Un attore è un'entità autonoma e puramente reattiva, che lavora solo quando riceve un messaggio. Per il resto del tempo, sta a riposo. Nel nostro

progetto, ogni componente fisico o logico è stato identificato come un'entità indipendente con un proprio stato interno:

- **UserInteraction** → attore che prende in input un PIN; attende messaggi da un listener. Quando viene inserito un input da tastiera lo riceve e invia il PIN ricevuto all'unità centrale.
- **Sensors** → Ogni attore di questo tipo rappresenta uno specifico sensore della casa (ad esempio PIR o sensore per le finestre). Ogni sensore comunica i movimenti rilevati all'unità centrale. La logica dei sensori prevede che ogni tot secondi venga rilevato un movimento. I sensori mandano messaggi a prescindere dallo stato del sistema, sarà poi l'unità centrale a decidere se ignorarli (in base allo stato in cui si trova).
- **Alarm** → attore che può ricevere messaggi per essere acceso e spento. Riceve due tipi di messaggi che sono "TurnOff()" e "TurnOn()".
Se un sensore manda un messaggio alla control unit quando si trova nello stato armed, quest'ultima manda "TurnOn()" all'Allarme e inizia un countdown. Se, durante il countdown, esso riceve dalla control unit un messaggio "TurnOff()" (PIN corretto) il conto alla rovescia si ferma. In caso contrario, manderà un messaggio "Alarm" alla control unit, che cambierà stato di conseguenza
- **Control Unit**: Unità centrale con diversi stati, gestisce le cose di conseguenza

Architettura e design del sistema

Il sistema è stato progettato seguendo una gerarchia ad albero, tipica del modello ad Attori per la supervisione e la gestione del ciclo di vita. Il componente centrale è lo **SmartHomeAlarmGuardian**, che funge da attore Root dell'applicazione. Al momento del setup, il guardiano istanzia (tramite `context.spawn`) tutti gli attori necessari: i diversi sensori, la sirena di allarme e il tastierino per l'interazione utente. Questa progettazione garantisce che nessun attore periferico comunichi direttamente con un altro, delegando l'intera logica al guardiano centrale.

La logica della **SmartHomeAlarmGuardian** è stata implementata come una macchina a stati finiti. Ogni stato (**Disarmed**, **ExitDelay**, **Armed**, **EntryDelay**, **Emergency**) è rappresentato da un metodo che restituisce un `Behavior[Command]`. Le transizioni di stato si verificano tramite l'invocazione del metodo successivo.

Abbiamo deciso di mantenere un basso livello di dipendenza tra gli attori. In effetti, gli attori **Sensor** e **UserInteraction** sono stati implementati in modo totalmente indipendente: il loro codice non contiene riferimenti allo **SmartHomeAlarmGuardian**. Per permettere al

guardiano di ricevere dati da questi componenti, abbiamo usato i Message Adapter all'interno del suo blocco setup. Il Message Adapter subentra nella fase di creazione dell'attore figlio, fornendogli un ActorRef specifico che si occupa di tradurre i messaggi. In particolare, intercetta i messaggi standard definiti nel protocollo (come MotionDetected dei sensori o TryInput del tastierino) e li converte nei tipi di comando interni che il guardiano è in grado di elaborare (rispettivamente DetectedMovement e InputEntered). Questo approccio rende il codice riusabile per eventuali estensioni di progetto.

- La sfida nella creazione dell'attore UserInteraction era implementare un input da tastiera asincrono. Siccome la lettura da tastiera è un'operazione sincrona e bloccante, abbiamo optato per una lettura da terminale classica (`StdIn.readLine()`), racchiusa però in una Future. Così facendo non è necessario ricevere immediatamente l'input, ma quando esso verrà inserito dall'utente verrà inviato un messaggio alla control unit con la stringa di digitata.
- L'attore Alarm d'altro canto è stato creato per togliere alla control unit la responsabilità della gestione del timer in entry delay, che risultava più complessa rispetto al resto del programma. Come accennato nella descrizione generale, il messaggio TurnOn() ricevuto dalla control unit attiverà un timer, al termine del quale l'attore manderà un messaggio a sé stesso, entrando in "EntryTimeout()". Stato in cui notifica la control unit, tramite l'attore AlarmStarting(), che è necessario il passaggio allo stato emergency. Il timer è però definito con una chiave univoca, questo per far sì che possa essere fermato quando l'attore riceve il comando TurnOff(), tramite la funzione "timers.cancel(SensorTimerKey)"
- Per quanto riguarda gli attori dei sensori si prevedono tre tipologie logiche: `PIRDoor`, `PIRLivingRoom` e `WindowSensor`, tuttavia il sistema crea quattro attori distinti: `sensorDoor`, `sensorLivingRoom`, `sensorWindow1`, `sensorWindow2`. Il loro protocollo è basato su due messaggi: Waiting e Detection, nel primo caso usa un timer con un delay randomico e al termine di questo viene rilevato un movimento, mentre nel secondo viene inviato il movimento al guardiano. Per questi attori è stato adottato il pattern dell'auto-messaggistica, dove l'attore invia questi messaggi alla propria mailbox tramite la reference context.self. Questa modalità permette all'attore di gestire i propri "cambi di stato" in modo asincrono e sicuro dal punto di vista della concorrenza. Infatti al termine del timer si invia il messaggio Detection per transitare nello stato di rilevazione del movimento, e dopo aver mandato il messaggio all'unità centrale si autoinvia il messaggio Waiting per ripristinare il suo comportamento di base.

Estensione bonus

Come estensione, il sistema richiede il supporto di suddivisione della casa in più zone (ad esempio, zona giorno e zona notte). L'idea è che ogni sensore sia associato ad una particolare zona, e durante l'attivazione l'utente possa scegliere se effettuare una copertura completa delle zone o una copertura parziale (in cui sono attive solo delle zone selezionate). In questo caso, i sensori nelle zone inattive vengono ignorati.

Per soddisfare i requisiti richiesti, nel protocollo condiviso del dominio abbiamo introdotto l'enum `zones` (es. `GroundFloor`, `UpperFloor`, `Garden`) per distinguere gli spazi fisici. Inoltre, è stato creato l'enum `modes` (es. `FullMode`, `NightMode`, `DayMode`) in cui ogni modalità incapsula la specifica lista di zone da mantenere attive.

Per permettere all'utente di selezionare la modalità di copertura richiesta, è stato creato uno stato intermedio tra `disarmed` e `exitDelay`, chiamato `chooseModality`. Questo stato associa a ciascuna modalità un numero, ed usa l'attore `userInteraction` allo stesso modo in cui viene usato nello stato `disarmed` per l'inserimento del pin corretto. Quando l'utente inserisce un numero corretto e preme invio la modalità sarà scelta e si passerà allo stato di `exit delay`.

Il controllo delle zone avviene all'interno dello stato `armed`. Quando il guardiano riceve una rilevazione di movimento, `armed` interroga la mappa usando l'id del sensore per scoprire in quale zona è avvenuto il movimento e verifica se tale zona è contenuta nella lista di zone attive (`activeMode`) passata come parametro:

```
if activeMode.contains(sensorForZone(sensorID)).
```

Se la zona è presente, il sistema invia un messaggio di accensione all'allarme e passa ad `EntryDelay`; se la zona non è presente, il messaggio viene semplicemente ignorato

Odds-and-Evens Game

Analisi dei requisiti

Il progetto richiede la modellazione e implementazione di una simulazione del gioco “Pari o Dispari” (Odds and Evens). Il gioco è strutturato come un torneo ad eliminazione diretta giocato da m players, dove il sistema verifica che m sia una potenza di due. Il gioco è ordinato in round sequenziali: all’inizio di ogni round, i giocatori devono essere accoppiati per scontrarsi in partite a due.

Per ogni match, i due giocatori fanno la loro scelta (pari o dispari), generano un numero (in maniera casuale) e il sistema ne calcola la somma per stabilire il vincitore. I vincitori del round passano al round successivo, e il torneo si conclude quando rimane un solo giocatore.

Le partite che compongono un singolo round del torneo devono essere eseguite in parallelo e in modo concorrente. L’architettura si deve basare sul paradigma del Message Passing (Passaggio di Messaggi). Ogni sincronizzazione deve avvenire tramite l’invio e la ricezione di messaggi sui canali.

Architettura e Design

L’esercizio è stato progettato, come richiesto, utilizzando il modello ad interazioni. Ciascun player è rappresentato in modo semplice con un id in una lista, e in ogni round diventa un’entità indipendente in modo simile ad un client: durante il round, infatti, il player genera la propria mossa (il numero casuale) e la invia sul canale assegnato, delegando qualsiasi computazione.

Per ogni round, la funzione principale main (che rappresenta il server) accoppia due player, fornendogli un canale privato. A questo punto viene creata una goroutine che, in modo asincrono, sceglie randomicamente un numero per ciascun player, inviando al canale privato l’id del player e la scelta (numero da 0 a 5) corrispondente. Successivamente, viene lanciata un’ulteriore goroutine, chiamata Match, che attende la ricezione delle informazioni di entrambi i player da un canale privato. Ottenute queste informazioni, viene scelto arbitrariamente quale tra i due player sarà pari o dispari, determinando di conseguenza il vincitore. Quest’ultimo verrà comunicato al server tramite un canale many-to-one condiviso tra gli altri match.

I vincitori sono quindi ricevuti dal server e inseriti in una lista che verrà usata per il round successivo. Ad ogni passo, viene dimezzato il numero di processi Player e Match allocati, fino ad ottenere un unico vincitore finale.

Per il passaggio dei messaggi sono stati impiegati i canali, inizializzati senza alcuna dimensione di buffer (es. `make(chan PlayerInfo)`). L'assenza di buffer implica che l'operazione di send effettuata da un giocatore si blocchi fino a quando il Match non esegue l'operazione di receive sullo stesso canale.

Canali:

- `MatchRoom([]chan PlayerInfo, m/2)` → Una lista di canali many-to-one, con numero di elementi pari alla metà dei player in ciascun round. Viene utilizzato dalla go routine Player in cui viene inviato il numero scelto, mentre la go routine Match riceve i due messaggi inviati
Player → MatchRoom | MatchRoom → Match
- `Results(chan int)` → Un singolo canale many-to-one che viene utilizzato da Match per inviare al "Server" i risultati delle partite
Match → Server

Il sistema è stato architettato per mitigare i rischi dati dalla gestione della sincronizzazione tra i processi, sfruttando le proprietà del message passing.

Per fare in modo che un round del torneo non termini prima che tutte le partite siano concluse, la funzione round lancia in parallelo tutte le goroutine necessarie (giocatori e match), il processo esegue un ciclo for per effettuare $m/2$ operazioni di receive sul canale results. Essendo la lettura un'operazione bloccante, il processo attende in modo sicuro che ogni match abbia calcolato il vincitore di tale partita. Solo quando tutti i risultati sono stati raccolti, il processo si sblocca, aggiorna la lista dei partecipanti e procede al round successivo.

L'utilizzo del Message Passing ha permesso di evitare l'uso di memoria condivisa: infatti, il sistema non usa variabili necessarie alla mutua esclusione. La consistenza dei dati è garantita in quanto l'accesso al contenuto dei canali è atomico. In effetti, se due processi provano a fare una send nello stesso istante sullo stesso canale, il canale non si corrompe: i messaggi si metteranno in fila ordinatamente da soli.

Conclusioni

Tramite il progetto implementato, abbiamo compreso l'efficacia del paradigma del Message Passing per far parlare tra di loro i processi. Abbiamo adottato Go, essendo uno dei linguaggi che usano il Message Passing sincrono puro, che ci ha permesso di non dover introdurre meccanismi di sincronizzazione esplicita e mutua esclusione (lock, semafori...). L'implementazione di questo paradigma risulta particolarmente significativa e potenzialmente vantaggiosa in ambiti specifici:

- Sistemi ad agenti indipendenti: Come dimostrato dal nostro torneo, quando si devono modellare entità autonome (utenti di una chat, giocatori di un videogioco multiplayer, sensori IoT) che operano in parallelo e necessitano di comunicare con un server centrale o tra di loro.
- Architetture a Pipeline: Quando un dato grezzo deve passare attraverso una serie di trasformazioni sequenziali (ad esempio, l'elaborazione di stream video o flussi di dati ETL).
- Progettazione verso i Sistemi Distribuiti: Utilizzare i canali e le goroutine in locale costringe il progettista a ragionare a messaggi. Questa modalità di pensiero è ottima come passaggio intermedio per poi arrivare alle architetture distribuite, dove semplicemente i messaggi per canali saranno sostituiti da messaggi di rete tra le due parti